Tugas Intelegensia Bisnis

Kategori 1: Data Preprocessing & ETL (Extract, Transform, Load)

Nama : **Muhammad Thifaal Dzaki**
NIM : **20230803103**

A. Soal

2. Konsolidasi Multi-Sumber Data Penjualan

- **Kasus:** Perusahaan memiliki data penjualan online (format CSV) dan data penjualan toko offline (dari database SQL).
- **Tugas Python:** Lakukan proses *merge* dan *concat* menggunakan Pandas untuk menyatukan kedua sumber data tersebut berdasarkan ID Produk dan Tanggal, lalu simpan ke format baru (Excel/CSV).

B. Kode Program dan Penjabaran Fungsi (soal2_konsolidasi_penjualan.py)

Berikut kode program yang saya buat. Kodenya saya bagi per fungsi sesuai urutan di dalam file, dan di bawah setiap bagian saya jelaskan kegunaan fungsi tersebut.

B.1 Docstring, Import, dan Konfigurasi

```
"""
Studi Kasus 2: Konsolidasi Multi-Sumber Data Penjualan
------------------------------------------------------
Kasus : Data penjualan ONLINE (file CSV) dan penjualan toko OFFLINE
        (database SQL) perlu disatukan.
Tugas : Proses concat & merge dengan Pandas berdasarkan ID Produk dan
        Tanggal, lalu simpan ke format baru (Excel/CSV).

Langkah:
  1. Muat data online (CSV) & offline (SQLite) -> buat data contoh jika tidak ada
  2. Samakan struktur kolom kedua sumber
  3. CONCAT : tumpuk semua transaksi online + offline (detail)
  4. MERGE  : gabungkan rekap online vs offline per (id_produk, tanggal)
  5. Simpan hasil ke Excel (multi-sheet) & CSV

Jalankan:
  python soal2_konsolidasi_penjualan.py                          # data contoh
  python soal2_konsolidasi_penjualan.py online.csv offline.db    # file sendiri
"""

import sqlite3
import os
import sys
from pathlib import Path

import numpy as np
import pandas as pd

# ---------------------------------------------------------------------------
# KONFIGURASI
# ---------------------------------------------------------------------------
FILE_ONLINE = "penjualan_online.csv"
FILE_OFFLINE = "penjualan_offline.db"
TABEL_OFFLINE = "penjualan_toko"
KUNCI = ["id_produk", "tanggal"]  # kunci penggabungan
OUTPUT_EXCEL = "konsolidasi_penjualan.xlsx"
OUTPUT_CSV_DETAIL = "konsolidasi_penjualan_detail.csv"
```

```
OUTPUT_CSV_REKAP = "konsolidasi_penjualan_rekap.csv"

PRODUK = {
    "P001": "Kopi Bubuk 200g",
    "P002": "Teh Celup 25s",
    "P003": "Gula Pasir 1kg",
    "P004": "Minyak Goreng 2L",
    "P005": "Beras Premium 5kg",
}
```

Bagian awal berisi docstring yang menerangkan studi kasus, langkah program, dan cara menjalankannya. Library yang dipakai yaitu **sqlite3** untuk database SQL, **os**, **sys**, dan **Path** untuk urusan folder, file, dan argumen, **numpy** untuk membuat data acak, serta **pandas** untuk mengolah tabel.

Di bagian konfigurasi ada nama file input, nama tabel database, dan tiga nama file output. **KUNCI** berisi kolom **id_produk** dan **tanggal** sebagai dasar penggabungan. Saya juga menambahkan **PRODUK**, yaitu daftar ID produk beserta namanya, misalnya **P001** untuk Kopi Bubuk 200g, supaya hasil akhirnya lebih mudah dibaca.

B.2 Fungsi buat_data_contoh()

```
# ---------------------------------------------------------------------------
# 1. MUAT DATA (atau buat data contoh)
# ---------------------------------------------------------------------------
def buat_data_contoh(path_csv: str, path_db: str, seed: int = 42) -> None:
    """Membuat file CSV (online) dan database SQLite (offline) contoh.

    Nama kolom kedua sumber sengaja dibuat berbeda, seperti kondisi nyata.
    """
    rng = np.random.default_rng(seed)
    tanggal = pd.date_range("2026-09-01", "2026-09-14", freq="D")
    harga = {"P001": 35_000, "P002": 12_000, "P003": 17_500,
             "P004": 38_000, "P005": 82_000}

    # --- Online (CSV) ---
    n = 120
    id_on = rng.choice(list(PRODUK), n)
    online = pd.DataFrame({
        "order_id": [f"ON-{i:04d}" for i in range(1, n + 1)],
        "product_id": id_on,
        "order_date": rng.choice(tanggal, n).astype("datetime64[ns]"),
        "qty": rng.integers(1, 6, n),
        "price": [harga[p] for p in id_on],
        "platform": rng.choice(["Tokopedia", "Shopee", "Website"], n),
    })
    online["order_date"] = online["order_date"].dt.strftime("%Y-%m-%d")
    online.to_csv(path_csv, index=False)

    # --- Offline (SQL) ---
    n = 100
    id_off = rng.choice(list(PRODUK), n)
    offline = pd.DataFrame({
        "no_struk": [f"OFF-{i:04d}" for i in range(1, n + 1)],
        "kode_produk": id_off,
        "tgl_transaksi": rng.choice(tanggal, n).astype("datetime64[ns]"),
        "jumlah": rng.integers(1, 8, n),
        "harga_satuan": [harga[p] for p in id_off],
        "cabang": rng.choice(["Jakarta", "Bekasi", "Tangerang"], n),
    })
    offline["tgl_transaksi"] = offline["tgl_transaksi"].dt.strftime("%Y-%m-%d")
    with sqlite3.connect(path_db) as conn:
        offline.to_sql(TABEL_OFFLINE, conn, if_exists="replace", index=False)

```

```
91        print(f"[INFO] Data contoh dibuat: {path_csv}, {path_db}")
```

Fungsi ini membuat data contoh kalau file sumber belum tersedia. Data yang dibuat ada dua, yaitu 120 transaksi online yang disimpan ke CSV dengan **to_csv()** dan 100 transaksi toko yang disimpan ke database SQLite dengan **to_sql()**. Angka acaknya dibuat dengan **np.random.default_rng(seed)** sehingga datanya selalu sama setiap kali dijalankan. Nama kolom data offline sengaja saya buat berbeda dari data online, misalnya **no_struk** dan **kode_produk**, supaya mirip dengan kondisi di lapangan.

B.3 Fungsi muat_online()

```
94    def muat_online(path: str) -> pd.DataFrame:
95        print(f"[INFO] Memuat data online (CSV): {path}")
96        return pd.read_csv(path)
```

Fungsi ini cukup sederhana, yaitu membaca data penjualan online dari file CSV dengan **pd.read_csv()**.

B.4 Fungsi muat_offline()

```
99    def muat_offline(path: str) -> pd.DataFrame:
100       print(f"[INFO] Memuat data offline (SQL): {path} -> tabel {TABEL_OFFLINE}")
101       with sqlite3.connect(path) as conn:
102           return pd.read_sql_query(f"SELECT * FROM {TABEL_OFFLINE}", conn)
```

Fungsi ini membaca data penjualan toko dari database. Koneksi dibuka dengan **sqlite3.connect()**, lalu isi tabel **penjualan_toko** diambil dengan **pd.read_sql_query()** sehingga langsung menjadi DataFrame.

B.5 Fungsi standarisasi_online()

```
105   # ------------------------------------------------------------------------
106   # 2. SAMAKAN STRUKTUR KOLOM
107   # ------------------------------------------------------------------------
108   def standarisasi_online(df: pd.DataFrame) -> pd.DataFrame:
109       df = df.rename(columns={
110           "order_id": "id_transaksi",
111           "product_id": "id_produk",
112           "order_date": "tanggal",
113           "qty": "jumlah",
114           "price": "harga_satuan",
115           "platform": "lokasi",
116       })
117       df["kanal"] = "Online"
118       return df
```

Fungsi ini mengganti nama kolom data online ke nama yang sudah saya tentukan, misalnya **order_id** menjadi **id_transaksi**, **product_id** menjadi **id_produk**, **order_date** menjadi **tanggal**, dan **platform** menjadi **lokasi**. Selain itu ditambahkan kolom **kanal** dengan isi "Online".

B.6 Fungsi standarisasi_offline()

```
121   def standarisasi_offline(df: pd.DataFrame) -> pd.DataFrame:
122       df = df.rename(columns={
123           "no_struk": "id_transaksi",
124           "kode_produk": "id_produk",
125           "tgl_transaksi": "tanggal",
126           "cabang": "lokasi",
127       })
128       df["kanal"] = "Offline"
129       return df
```

Fungsi ini sama seperti sebelumnya tetapi untuk data offline. Kolom **no_struk** diganti menjadi **id_transaksi**, **kode_produk** menjadi **id_produk**, **tgl_transaksi** menjadi **tanggal**, dan **cabang** menjadi **lokasi**, lalu ditambahkan kolom **kanal** berisi "Offline". Setelah dua fungsi ini dijalankan, nama kolom kedua data sudah sama.

B.7 Fungsi rapikan_tipe()

```
132   def rapikan_tipe(df: pd.DataFrame) -> pd.DataFrame:
133       df = df.copy()
134       df["id_produk"] = df["id_produk"].astype(str).str.strip().str.upper()
135       df["tanggal"] = pd.to_datetime(df["tanggal"]).dt.normalize()
136       df["jumlah"] = pd.to_numeric(df["jumlah"])
137       df["harga_satuan"] = pd.to_numeric(df["harga_satuan"])
138       df["total"] = df["jumlah"] * df["harga_satuan"]
139       return df
```

Walaupun nama kolomnya sudah sama, tipe datanya belum tentu sama. Misalnya ID produk bisa saja ada spasi atau huruf kecil, dan tanggal masih berupa teks. Fungsi ini merapikan hal tersebut. **id_produk** dibersihkan dengan **str.strip()** dan dijadikan huruf besar dengan **str.upper()**, **tanggal** diubah ke tipe tanggal dengan **pd.to_datetime()**, sedangkan **jumlah** dan **harga_satuan** diubah ke angka dengan **pd.to_numeric()**. Di akhir dihitung kolom **total**.

B.8 Fungsi gabung_concat()

```
142   # ------------------------------------------------------------------------
143   # 3. CONCAT -> data detail gabungan
144   # ------------------------------------------------------------------------
145   def gabung_concat(online: pd.DataFrame, offline: pd.DataFrame) -> pd.DataFrame:
146       kolom = ["id_transaksi", "kanal", "lokasi", "id_produk", "tanggal",
147                "jumlah", "harga_satuan", "total"]
148       detail = pd.concat([online[kolom], offline[kolom]], ignore_index=True)
149       detail.insert(4, "nama_produk", detail["id_produk"].map(PRODUK))
150       return detail.sort_values(KUNCI + ["kanal"]).reset_index(drop=True)
```

Fungsi ini menjalankan proses concat. Urutan kolom kedua data disamakan dulu, kemudian digabung dengan **pd.concat()** sehingga semua transaksi online dan offline ada dalam satu tabel detail. Setelah itu ditambahkan kolom **nama_produk** yang diambil dari **PRODUK**, lalu tabel diurutkan berdasarkan ID Produk, Tanggal, dan kanal.

B.9 Fungsi rekap_per_kunci()

```
153   # ------------------------------------------------------------------------
154   # 4. MERGE -> rekap online vs offline per (id_produk, tanggal)
155   # ------------------------------------------------------------------------
156   def rekap_per_kunci(df: pd.DataFrame, akhiran: str) -> pd.DataFrame:
157       return (df.groupby(KUNCI, as_index=False)
158                 .agg(jumlah=("jumlah", "sum"), total=("total", "sum"))
159                 .rename(columns={"jumlah": f"qty_{akhiran}",
160                                  "total": f"omzet_{akhiran}"}))
```

Fungsi ini dipakai sebelum merge. Data dari satu kanal direkap dengan **groupby()** sehingga setiap pasangan ID Produk dan Tanggal hanya punya satu baris berisi total jumlah dan total omzet. Nama kolomnya diberi akhiran sesuai kanal, contohnya **qty_online** dan **omzet_online**.

B.10 Fungsi gabung_merge()

```
163   def gabung_merge(online: pd.DataFrame, offline: pd.DataFrame) -> pd.DataFrame:
164       rekap = pd.merge(
165           rekap_per_kunci(online, "online"),
166           rekap_per_kunci(offline, "offline"),
167           on=KUNCI,
```

```
168           how="outer",       # pertahankan produk/tanggal yang hanya ada di satu kanal
169           indicator="sumber",
170       )
171       rekap["sumber"] = rekap["sumber"].map({
172           "left_only": "Hanya Online", "right_only": "Hanya Offline", "both": "Keduanya",
173       })
174       kolom_angka = ["qty_online", "omzet_online", "qty_offline", "omzet_offline"]
175       rekap[kolom_angka] = rekap[kolom_angka].fillna(0).astype("int64")
176       rekap["qty_total"] = rekap["qty_online"] + rekap["qty_offline"]
177       rekap["omzet_total"] = rekap["omzet_online"] + rekap["omzet_offline"]
178       rekap.insert(1, "nama_produk", rekap["id_produk"].map(PRODUK))
179       return rekap.sort_values(KUNCI).reset_index(drop=True)
```

Fungsi ini menjalankan proses merge. Rekap online dan rekap offline digabung dengan **pd.merge()** berdasarkan **KUNCI**. Saya memakai **how="outer"** agar produk yang pada tanggal tertentu hanya terjual di salah satu kanal tetap ikut masuk. Parameter **indicator="sumber"** dipakai untuk mencatat asal setiap baris, yang kemudian saya ubah menjadi keterangan "Keduanya", "Hanya Online", atau "Hanya Offline".

Karena outer join menghasilkan nilai kosong pada kanal yang tidak ada transaksinya, nilai tersebut diisi 0 dengan **fillna(0)**. Terakhir dihitung **qty_total** dan **omzet_total**, lalu ditambahkan kolom **nama_produk**.

B.11 Fungsi validasi()

```
182   # ------------------------------------------------------------------------
183   # 5. VALIDASI & SIMPAN
184   # ------------------------------------------------------------------------
185   def validasi(online, offline, detail, rekap) -> None:
186       assert len(detail) == len(online) + len(offline), "Jumlah baris concat tidak cocok!"
187       assert not rekap.duplicated(KUNCI).any(), "Kunci (id_produk, tanggal) duplikat!"
188       assert rekap["omzet_total"].sum() == detail["total"].sum(), "Omzet merge != concat!"
```

Fungsi ini saya buat untuk mengecek apakah hasil penggabungan sudah benar. Ada tiga pengecekan. Jumlah baris hasil concat harus sama dengan jumlah data online ditambah offline, pasangan **id_produk** dan **tanggal** di hasil merge tidak boleh dobel, dan total omzet hasil merge harus sama dengan total omzet hasil concat. Kalau ada yang tidak sesuai, **assert** akan menghentikan program dan menampilkan pesan kesalahannya.

B.12 Fungsi ringkasan_produk()

```
191   def ringkasan_produk(detail: pd.DataFrame) -> pd.DataFrame:
192       return (detail.pivot_table(index=["id_produk", "nama_produk"], columns="kanal",
193                                  values="total", aggfunc="sum", fill_value=0)
194                     .assign(Total=lambda x: x.sum(axis=1))
195                     .reset_index())
```

Fungsi ini membuat ringkasan omzet per produk. Dengan **pivot_table()**, omzet online dan offline setiap produk ditampilkan berdampingan, lalu **assign()** menambahkan kolom **Total**. Ringkasan ini tidak diminta langsung oleh soal, tetapi membantu untuk melihat produk mana yang paling banyak menghasilkan omzet.

B.13 Fungsi simpan()

```
198   def simpan(detail, rekap, per_produk) -> None:
199       for df in (detail, rekap):
200           df["tanggal"] = df["tanggal"].dt.strftime("%Y-%m-%d")
201
202       detail.to_csv(OUTPUT_CSV_DETAIL, index=False)
203       rekap.to_csv(OUTPUT_CSV_REKAP, index=False)
204       with pd.ExcelWriter(OUTPUT_EXCEL, engine="openpyxl") as writer:
```

```
205           detail.to_excel(writer, sheet_name="Detail (concat)", index=False)
206           rekap.to_excel(writer, sheet_name="Rekap (merge)", index=False)
207           per_produk.to_excel(writer, sheet_name="Ringkasan Produk", index=False)
208       print(f"\n[OK] Disimpan: {OUTPUT_EXCEL}, {OUTPUT_CSV_DETAIL}, {OUTPUT_CSV_REKAP}")
```

Fungsi ini menyimpan hasil ke format baru sesuai permintaan soal. Tanggal ditulis dulu dalam format **YYYY-MM-DD** dengan **dt.strftime()**. Tabel detail dan tabel rekap disimpan ke dua file CSV dengan **to_csv()**, lalu ketiganya disimpan ke satu file Excel dengan **pd.ExcelWriter**, yaitu sheet Detail (concat), Rekap (merge), dan Ringkasan Produk.

B.14 Fungsi main()

```
211   # ---------------------------------------------------------------------------
212   # MAIN
213   # ---------------------------------------------------------------------------
214   def main() -> None:
215       os.chdir(Path(__file__).resolve().parent)  # baca/simpan file di folder ini
216       path_csv = sys.argv[1] if len(sys.argv) > 1 else FILE_ONLINE
217       path_db = sys.argv[2] if len(sys.argv) > 2 else FILE_OFFLINE
218       if not (Path(path_csv).exists() and Path(path_db).exists()):
219           print("[INFO] File sumber tidak ditemukan -> membuat data contoh.")
220           buat_data_contoh(path_csv, path_db)
221
222       online = rapikan_tipe(standarisasi_online(muat_online(path_csv)))
223       offline = rapikan_tipe(standarisasi_offline(muat_offline(path_db)))
224       print(f"\nTransaksi online : {len(online):,} baris")
225       print(f"Transaksi offline: {len(offline):,} baris")
226
227       detail = gabung_concat(online, offline)
228       rekap = gabung_merge(online, offline)
229       validasi(online, offline, detail, rekap)
230       per_produk = ringkasan_produk(detail)
231
232       print("\n=== HASIL CONCAT (5 baris pertama) ===")
233       print(detail.head().to_string(index=False))
234       print(f"Total baris detail: {len(detail):,}")
235
236       print("\n=== HASIL MERGE per (id_produk, tanggal) (5 baris pertama) ===")
237       print(rekap.head().to_string(index=False))
238       print(f"Total baris rekap : {len(rekap):,}")
239       print(f"\nSebaran sumber data:\n{rekap['sumber'].value_counts().to_string()}")
240
241       print("\n=== RINGKASAN OMZET PER PRODUK (Rp) ===")
242       print(per_produk.to_string(index=False))
243       print(f"\nTotal omzet gabungan: Rp {detail['total'].sum():,.0f}")
244
245       simpan(detail, rekap, per_produk)
```

Fungsi **main()** mengatur jalannya program dari awal sampai akhir. Folder kerja dipindahkan dulu dengan **os.chdir()**. Program bisa menerima nama file dari **sys.argv**, dan kalau filenya tidak ditemukan maka **buat_data_contoh()** dipanggil untuk membuat data contoh.

Setelah itu data online dan offline dibaca, kolomnya disamakan, dan tipe datanya dirapikan. Data kemudian diproses dengan **gabung_concat()** dan **gabung_merge()**, dicek dengan **validasi()**, dan dibuatkan ringkasan dengan **ringkasan_produk()**. Hasilnya ditampilkan di terminal lalu disimpan dengan **simpan()**.

B.15 Bagian if __name__

```
248   if __name__ == "__main__":
249       main()
```

Baris terakhir ini membuat fungsi **main()** hanya dijalankan kalau file dibuka langsung lewat terminal. Jadi kalau file ini di-import dari program lain, prosesnya tidak langsung berjalan.

C. Hasil / Jawaban

Program saya jalankan dengan perintah **python soal2_konsolidasi_penjualan.py** dari folder tempat file program berada, dan output yang muncul di terminal adalah sebagai berikut.

```
[INFO] Memuat data online (CSV): penjualan_online.csv
[INFO] Memuat data offline (SQL): penjualan_offline.db -> tabel penjualan_toko

Transaksi online : 120 baris
Transaksi offline: 100 baris

=== HASIL CONCAT (5 baris pertama) ===
id_transaksi   kanal    lokasi id_produk     nama_produk    tanggal  jumlah  harga_satuan  total
     ON-0018  Online   Website      P001 Kopi Bubuk 200g 2026-09-01       1         35000  35000
    OFF-0094 Offline   Jakarta      P001 Kopi Bubuk 200g 2026-09-02       6         35000 210000
     ON-0007  Online    Shopee      P001 Kopi Bubuk 200g 2026-09-02       4         35000 140000
     ON-0052  Online Tokopedia      P001 Kopi Bubuk 200g 2026-09-02       2         35000  70000
    OFF-0043 Offline   Jakarta      P001 Kopi Bubuk 200g 2026-09-03       6         35000 210000
Total baris detail: 220

=== HASIL MERGE per (id_produk, tanggal) (5 baris pertama) ===
id_produk     nama_produk    tanggal  qty_online  omzet_online  qty_offline  omzet_offline        sumber  qty_total  omzet_total
     P001 Kopi Bubuk 200g 2026-09-01           1         35000            0              0  Hanya Online          1        35000
     P001 Kopi Bubuk 200g 2026-09-02           6        210000            6         210000      Keduanya         12       420000
     P001 Kopi Bubuk 200g 2026-09-03           6        210000           13         455000      Keduanya         19       665000
     P001 Kopi Bubuk 200g 2026-09-04           4        140000            4         140000      Keduanya          8       280000
     P001 Kopi Bubuk 200g 2026-09-05           0             0            9         315000 Hanya Offline          9       315000
Total baris rekap : 66

Sebaran sumber data:
sumber
Keduanya         49
Hanya Offline     9
Hanya Online      8

=== RINGKASAN OMZET PER PRODUK (Rp) ===
id_produk       nama_produk  Offline  Online    Total
     P001   Kopi Bubuk 200g  3640000 2100000  5740000
     P002     Teh Celup 25s  1056000  576000  1632000
     P003    Gula Pasir 1kg  1330000 1435000  2765000
     P004  Minyak Goreng 2L  2128000 3458000  5586000
     P005 Beras Premium 5kg  8446000 5248000 13694000

Total omzet gabungan: Rp 29,417,000

[OK] Disimpan: konsolidasi_penjualan.xlsx, konsolidasi_penjualan_detail.csv, konsolidasi_penjualan_rekap.csv
```

Gambar 1. Output program di terminal.

Hasil concat berjumlah 220 baris, terdiri dari 120 transaksi online dan 100 transaksi offline, dan sudah dilengkapi nama produk serta lokasi penjualannya. Hasil merge berjumlah 66 baris. Dari jumlah tersebut, 49 kombinasi produk dan tanggal terjual di kedua kanal, 9 hanya terjual di offline, dan 8 hanya terjual di online.

Ketiga pengecekan di fungsi validasi lolos, jadi tidak ada data yang hilang atau dobel dan total omzet merge sama dengan concat. Produk dengan omzet paling besar adalah Beras Premium 5kg sebesar Rp 13.694.000, dan penjualannya lebih banyak di toko offline. Total omzet gabungan adalah Rp 29.417.000. File yang dihasilkan yaitu **konsolidasi_penjualan.xlsx** dengan tiga sheet, **konsolidasi_penjualan_detail.csv**, dan **konsolidasi_penjualan_rekap.csv**.

id_transaksi	kanal	lokasi	id_produk	nama_produk	tanggal	jumlah	harga_satuan	total
ON-0018	Online	Website	P001	Kopi Bubuk 200g	2026-09-01	1	35000	35000
OFF-0094	Offline	Jakarta	P001	Kopi Bubuk 200g	2026-09-02	6	35000	210000
ON-0007	Online	Shopee	P001	Kopi Bubuk 200g	2026-09-02	4	35000	140000
ON-0052	Online	Tokopedia	P001	Kopi Bubuk 200g	2026-09-02	2	35000	70000
OFF-0043	Offline	Jakarta	P001	Kopi Bubuk 200g	2026-09-03	6	35000	210000
OFF-0084	Offline	Tangerang	P001	Kopi Bubuk 200g	2026-09-03	7	35000	245000
ON-0058	Online	Tokopedia	P001	Kopi Bubuk 200g	2026-09-03	3	35000	105000
ON-0104	Online	Shopee	P001	Kopi Bubuk 200g	2026-09-03	3	35000	105000
OFF-0038	Offline	Jakarta	P001	Kopi Bubuk 200g	2026-09-04	4	35000	140000
ON-0023	Online	Website	P001	Kopi Bubuk 200g	2026-09-04	2	35000	70000
ON-0069	Online	Tokopedia	P001	Kopi Bubuk 200g	2026-09-04	2	35000	70000
OFF-0052	Offline	Jakarta	P001	Kopi Bubuk 200g	2026-09-05	3	35000	105000
OFF-0058	Offline	Jakarta	P001	Kopi Bubuk 200g	2026-09-05	3	35000	105000
OFF-0066	Offline	Jakarta	P001	Kopi Bubuk 200g	2026-09-05	3	35000	105000
OFF-0048	Offline	Tangerang	P001	Kopi Bubuk 200g	2026-09-06	1	35000	35000
ON-0045	Online	Shopee	P001	Kopi Bubuk 200g	2026-09-06	3	35000	105000
ON-0074	Online	Tokopedia	P001	Kopi Bubuk 200g	2026-09-06	2	35000	70000
OFF-0042	Offline	Bekasi	P001	Kopi Bubuk 200g	2026-09-07	7	35000	245000
OFF-0053	Offline	Bekasi	P001	Kopi Bubuk 200g	2026-09-07	7	35000	245000
OFF-0064	Offline	Jakarta	P001	Kopi Bubuk 200g	2026-09-07	2	35000	70000

Gambar 2. Isi file output konsolidasi_penjualan_detail.csv (20 baris pertama dari 220 baris).

id_produk	nama_produk	tanggal	qty_online	omzet_online	qty_offline	omzet_offline	sumber	qty_total	omzet_total
P001	Kopi Bubuk 200g	2026-09-01	1	35000	0	0	Hanya Online	1	35000
P001	Kopi Bubuk 200g	2026-09-02	6	210000	6	210000	Keduanya	12	420000
P001	Kopi Bubuk 200g	2026-09-03	6	210000	13	455000	Keduanya	19	665000
P001	Kopi Bubuk 200g	2026-09-04	4	140000	4	140000	Keduanya	8	280000
P001	Kopi Bubuk 200g	2026-09-05	0	0	9	315000	Hanya Offline	9	315000
P001	Kopi Bubuk 200g	2026-09-06	5	175000	1	35000	Keduanya	6	210000
P001	Kopi Bubuk 200g	2026-09-07	15	525000	16	560000	Keduanya	31	1085000
P001	Kopi Bubuk 200g	2026-09-09	9	315000	22	770000	Keduanya	31	1085000
P001	Kopi Bubuk 200g	2026-09-10	4	140000	5	175000	Keduanya	9	315000
P001	Kopi Bubuk 200g	2026-09-11	2	70000	4	140000	Keduanya	6	210000
P001	Kopi Bubuk 200g	2026-09-12	0	0	5	175000	Hanya Offline	5	175000
P001	Kopi Bubuk 200g	2026-09-13	6	210000	6	210000	Keduanya	12	420000
P001	Kopi Bubuk 200g	2026-09-14	2	70000	13	455000	Keduanya	15	525000
P002	Teh Celup 25s	2026-09-02	9	108000	13	156000	Keduanya	22	264000
P002	Teh Celup 25s	2026-09-04	7	84000	1	12000	Keduanya	8	96000
P002	Teh Celup 25s	2026-09-05	5	60000	1	12000	Keduanya	6	72000
P002	Teh Celup 25s	2026-09-06	6	72000	6	72000	Keduanya	12	144000
P002	Teh Celup 25s	2026-09-07	10	120000	1	12000	Keduanya	11	132000
P002	Teh Celup 25s	2026-09-08	1	12000	10	120000	Keduanya	11	132000
P002	Teh Celup 25s	2026-09-09	4	48000	7	84000	Keduanya	11	132000

Gambar 3. Isi file output konsolidasi_penjualan_rekap.csv (20 baris pertama dari 66 baris).

Kesimpulan:

Program ini berhasil menyatukan data penjualan online dari CSV dan data toko offline dari database SQL dengan concat dan merge berdasarkan ID Produk dan Tanggal, lalu menyimpannya ke Excel dan CSV. Selain menjawab soal, program juga merapikan tipe data supaya penggabungan tidak gagal, bisa dijalankan dengan file sendiri maupun data contoh, mencatat asal data dari setiap baris hasil merge, dan memiliki pengecekan otomatis. Ringkasan omzet per produk juga bisa dipakai untuk melihat produk yang paling laku.